Unidad 101: Arquitectura del Sistema

- 101.1: Configurar y determinar el HW necesario (2)
- 101.2: Conocer el arranque del SO. (3)
- 101.3 Cambiar los niveles de Ejecución: Apagar y reiniciar el sistema (2)

101.1: Configurar y determinar el HW necesario (Peso: 2)

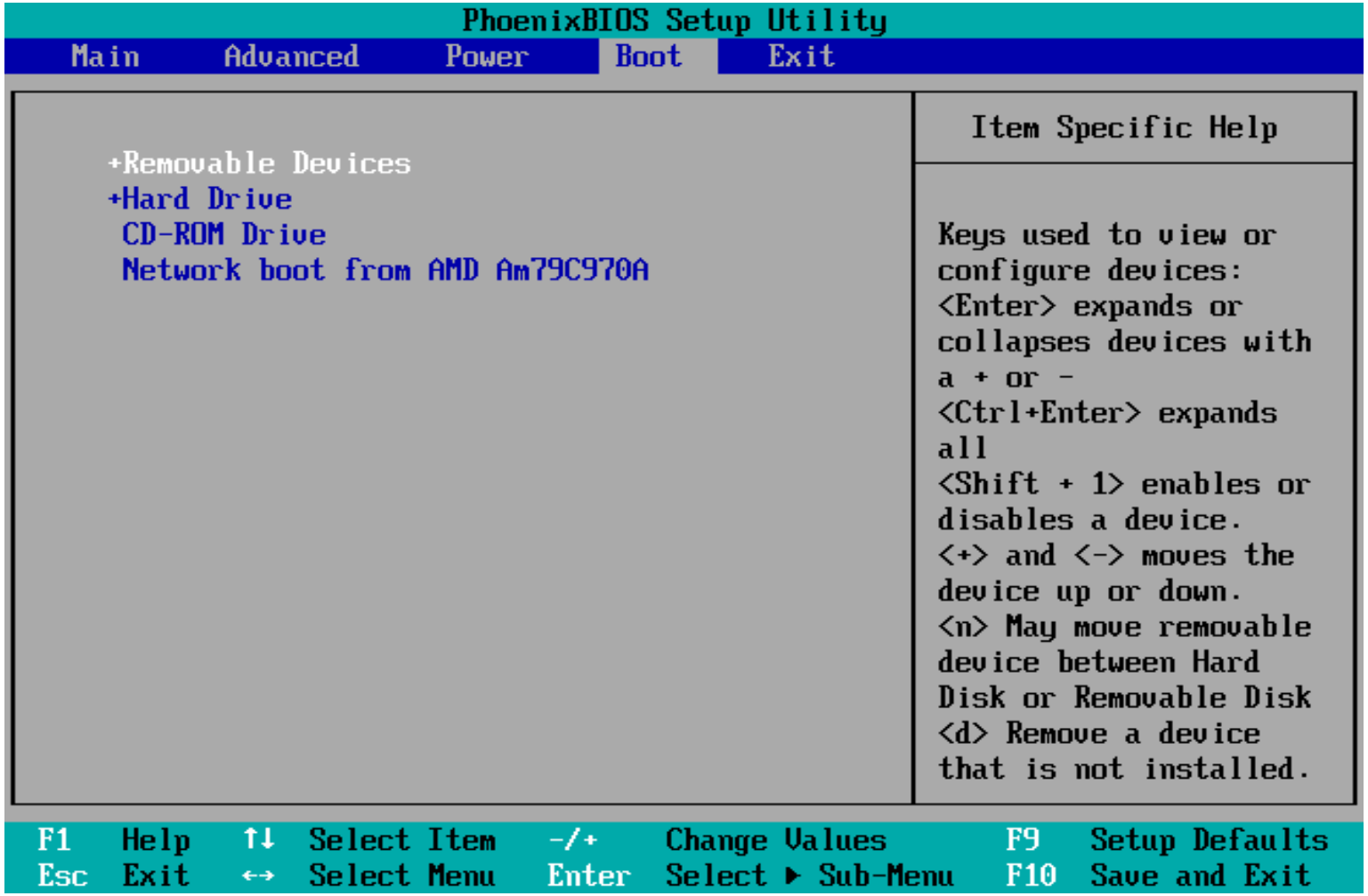
Conociendo un poco la máquina

Un Sistema Operativo (SO) es el software que hace el enlace entre el usuario y el Hardware

Mayoría de dispositivos HW contienen un firmware

Arquitecturas 8086 el firmware es **BIOS** (Basic Input/Output System)

101.1: Configurar y determinar el HW necesario (Peso: 2)



101.1: Configurar y determinar el HW necesario (Peso: 2)

La BIOS (Basic Input Output System) es un software de interfaz entre el hardware y el software a un nivel muy básico.Proporciona el conjunto de las instrucciones básicas utilizadas por el sistema operativo. Provee el nivel de interfaz más bajo a los drivers y periféricos.

La BIOS está presente en una memoria EEPROM (Electrical Erasable Programmable ReadOnly Memory) del ordenador. Cuando se enciende el ordenador, o tras un reseteado, se manda una señal llamada al microprocesador. Éste activa entonces la ejecución de la BIOS.

101.1: Configurar y determinar el HW necesario (Peso: 2)

La BIOS efectúa un autochequeo del encendido **(POST)**, luego busca los periféricos, en particular los utilizados para iniciar el sistema. A continuación se almacena la información relativa al hardware de manera permanente en una pequeña memoria CMOS alimentada por una batería. Al final de proceso, se selecciona el periférico de inicio.

La BIOS lee y ejecuta el primer sector físico del soporte físico de inicio. Suele tratarse de los **512 primeros bytes del primer disco duro (el MBR**) o de la partición activa (la PBR)

101.1: Configurar y determinar el HW necesario (Peso: 2)

COMANDOS BÁSICOS

cat

more/less

man

ls

cd

101.1: Configurar y determinar el HW necesario (Peso: 2)

/proc

El sistema virtual /proc está lleno de información sobre Procesos y Hardware.

/proc/PID → info sobre procesos

/proc/sys → opciones dinamicas del kernel

/proc/cpuinfo → CPU

/proc/meminfo → memoria

/proc/version → version del sistema

101.1: Configurar y determinar el HW necesario (Peso: 2)

Interrupciones

IRQs (Interrupt request): evento o señal enviada a la CPU para ejecutarse.

Cuando un periférico, (por ejemplo una impresora) u otro dispositivo hardware (e.g. una tarjeta de sonido), necesitan "comunicarse" con la CPU utilizan una líneas de notificación preestablecidas denominadas, los IRQ’s

101.1: Configurar y determinar el HW necesario (Peso: 2)

Señales actuales ejecutadas en el SO Linux las podemos ver en el fichero **/proc/interrupts**

```
# cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3
0:      33932      33887      33870      33887   IO-APIC-edge      timer
1:          2          0          1          0   IO-APIC-edge      i8042
 8:         20         21         20         17   IO-APIC-edge      rtc0
 9:          0          0          0          0   IO-APIC-fasteoi   acpi
 12:         4          0          1          0   IO-APIC-edge      i8042
 14:         0          0          0          0   IO-APIC-edge      ide0
15:          0          0          0          0   IO-APIC-edge      ide1
 16:         0          0          0          0   IO-APIC-fasteoi   uhci_hcd:usb5
 18:         0          0          0          0   IO-APIC-fasteoi   uhci_hcd:us
```

101.1: Configurar y determinar el HW necesario (Peso: 2)

I/O Addresses: localizaciones en memoria reservadas para la comunicación entre CPU y dispositivos HW. cat **/proc/ioports**

```
# cat /proc/ioports
0000-001f: dma1
0020-0021: pic1
0040-0043: timer0
0050-0053: timer1
0060-006f: keyboard
0070-0077: rtc
0080-008f: dma page reg
00a0-00a1: pic2
00c0-00df: dma2
00f0-00ff: fpu
0290-0297: pnp 00:06
```

101.1: Configurar y determinar el HW necesario (Peso: 2)

DMA: Direct Memory Addressing. Se tratan los accesos de memoria a diferentes dispositivos. cat**/proc/dma**

cat /proc/dma

4: cascada

Interrupciones del Sistema. cat**/proc/interrupts**

```
# cat /proc/interrupts
CPU0 CPU1
0: 692294 559242 IO-APIC-edge timer
1: 1 1 IO-APIC-edge i8042
8: 1 0 IO-APIC-edge rtc
9: 0 0 IO-APIC-fasteoi acpi
12: 0 4 IO-APIC-edge i8042
16: 591 430773 IO-APIC-fasteoi ahci, uhci_hcd:usb1,
ohci1394, nvidia
17: 39 15025 IO-APIC-fasteoi libata
18: 285 39829 IO-APIC-fasteoi ehci_hcd:usb3,
```

101.1: Configurar y determinar el HW necesario (Peso: 2)

Pregunta de exámen:

Which files contain essential system information such as IRQs, direct memory access channels, and I/O addresses? (Select three.)

A. /proc/ioports

B. /proc/ioaddresses

C. /proc/dma

D. /proc/interrupts

E. /proc/hardware

101.1: Configurar y determinar el HW necesario (Peso: 2)

Trabajando con dispositivos

Los dispositivos **Hotplug** y **Coldplug** se diferencian en que se pueden conectar estando el servidor encendido o apagado.

Linux nos ayuda con difentes utilidades o conceptos a la hora de tratar temas de hardware

¿EJEMPLOS?

101.1: Configurar y determinar el HW necesario (Peso: 2)

Elementos y directorios HW

Sysfs: sistema virtual montado en **/sys** que nos proporciona información sobre procesos de sistema (Evolución del /proc)

Udev: Linux crea nodos de dispositivos como archivos (**/dev**).Estos archivos se almacenan en un sistema de archivos virtual.

101.1: Configurar y determinar el HW necesario (Peso: 2)

Las tarjetas de expansión: PCI

¿Cómo vemos las tarjetas en nuestro SO?

lspci *opción*

lspci

00:00.0 Host bridge: Intel Corporation Atom Processor D2xxx/N2xxx DRAM Controller (rev 03)

00:02.0 VGA compatible controller: Intel Corporation Atom Processor D2xxx/N2xxx Integrated Graphics Controller (rev 09)

00:1c.0 PCI bridge: Intel Corporation N10/ICH 7 Family PCI Express Port 1 (rev 02)

00:1d.0 USB controller: Intel Corporation N10/ICH 7 Family USB UHCI Controller #1 (rev 02)

00:1d.1 USB controller: Intel Corporation N10/ICH 7 Family USB UHCI Controller #2 (rev 02)

00:1d.2 USB controller: Intel Corporation N10/ICH 7 Family USB UHCI Controller #3 (rev 02)

00:1d.3 USB controller: Intel Corporation N10/ICH 7 Family USB UHCI Controller #4 (rev 02)

101.1: Configurar y determinar el HW necesario (Peso: 2)

Gestión de Módulos

El hardware el Linux es gestionado por drivers del kernel. Éstos vienen de los **módulos del kernel** (almacenados en /lib/modules)

lsmod *[opción]*

```
# lsmod
Module                  Size  Used by
lock_dlm               51425  0
gfs2                  526829  1 lock_dlm
dlm                   159553  11 lock_dlm
configfs               62301  2 dlm
iptable_filter         36161  0
ip_tables              55329  1 iptable_filter
x_tables               50377  1 ip_tables
mptctl                 63817  1
mptbase               113637  1 mptctl
ipmi_watchdog          52641  0
```

101.1: Configurar y determinar el HW necesario (Peso: 2)

Gestión de Módulos

Nosotros mismos podemos cargar (añadir) **módulos del kernel**. Tienen **extensión *.ko***

Ejemplo:

insmod /lib/modules/2.6.26/kernel/drivers/floppy.ko

101.1: Configurar y determinar el HW necesario (Peso: 2)

Gestión de Módulos

Nosotros mismos podemos eliminar (quitar) módulos del kernel.

Ejemplo:

rmmod floppy

101.1: Configurar y determinar el HW necesario (Peso: 2)

Gestión de Módulos

Algunos módulos dependen de otros módulos para añadirlos. Si un módulo del que depende no está. FALLARÁ.

modprobe: aplicación que nos facilita las dependencias de módulos

Ejemplo:

modprobe *floppy*

101.1: Configurar y determinar el HW necesario (Peso: 2)

Gestión de Módulos

modprobe se basa en un archivo de configuración. **/etc/modprobe.conf**

Es posible que también nos encontremos con diferentes configuraciones dentro del directorio

/etc/modprobe.d/

101.1: Configurar y determinar el HW necesario (Peso: 2)

Interfaces

USB es un protocolo para transferir datos entre dispositivos.

El comando **lsusb** nos muestra la lista de dispositivos USB conectados a nuestro SO

101.1: Configurar y determinar el HW necesario (Peso: 2)

Pregunta de exámen:

What tool would you use to disable a motherboard's sound hardware if you don't want to use it?

A. The firmware

B. The alsactl utility

C. The lsmod command

D. The lspci program

E. None of the above; onboard sound devices can't be disabled

101.2: Conocer el arranque del SO (Peso: 3)

Pasos para el arranque

Debemos tener en cuenta tres aspectos importantes:

BIOS

Bootloader

Kernel

101.2: Conocer el arranque del SO (Peso: 3)

BIOS es el primer programa en ejecutarse. Se ejecuta desde la ROM y tiene su propio SO independendiente

Posteriormente se ejecuta un **POST** que realiza un test independiente de HW para poder arrancar.

Luego es turno del **SO** desde un dispositivo de almacenamiento (Busca MBR)

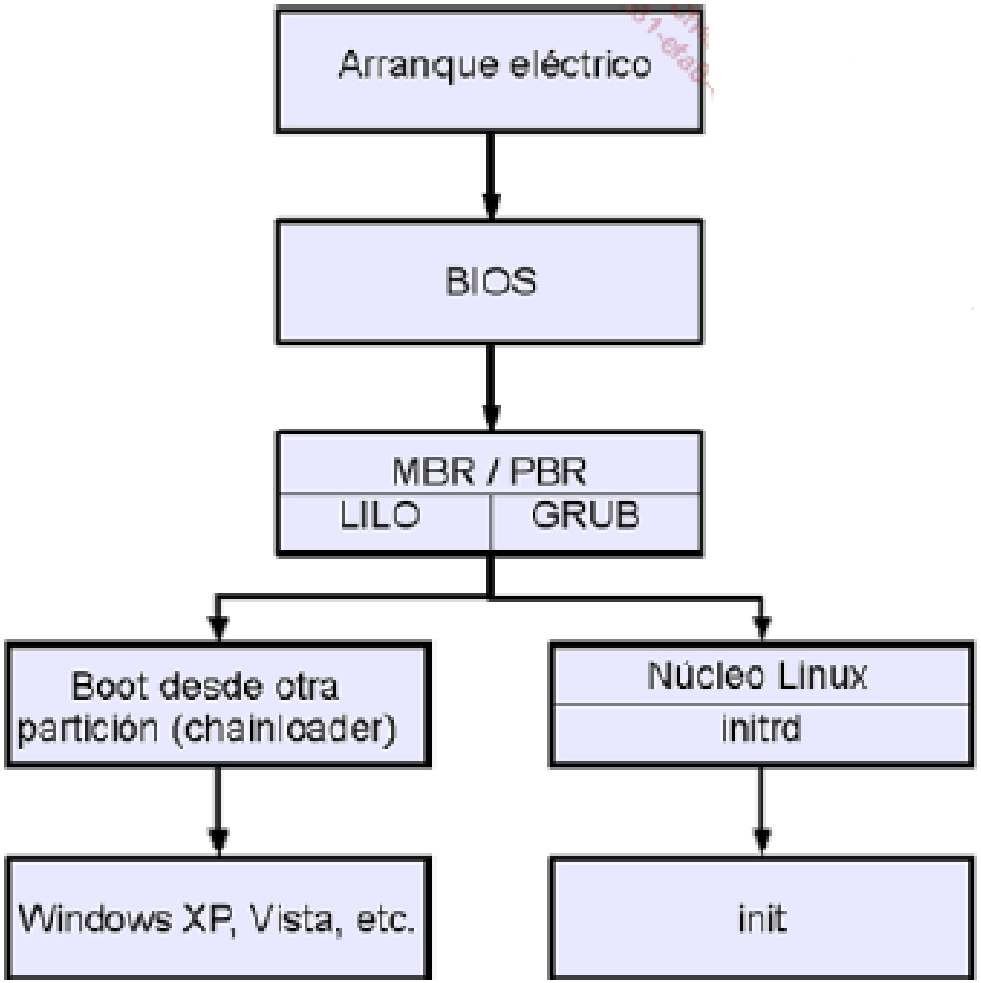
MBR lanza el bootloader el cual carga el kernel del SO con el *initial-ramdisk*

El kernel cargado se suele encontrar sobre **/boot/vmlinuz/**

A partir de aquí se ejecuta el programa **linuxrc** (se ejecuta durante el inicio del Kernel). Éste ofrece una lista de controladores disponibles

Man Initrd

101.2: Conocer el arranque del SO (Peso: 3)



101.2: Conocer el arranque del SO (Peso: 3)

initial ramdisk es desmontada y **el Sistema de Archivos Linux es montado**

El Kernel ejecuta los niveles **init**

¿Y qué es eso de los niveles init?

Niveles de ejecución para los distintos Servicios

101.2: Conocer el arranque del SO (Peso: 3)

Depurando el arranque

¿Cómo depuramos mensajes del Kernel?

1./var/log/messages

2.dmesg: imprime señales del kernel al arranque del sistema o cuando recibe señales de dispositivos.

101.3: Niveles de Ejecución (Peso: 3)

Niveles init o de ejecución

Un nivel de ejecución, o runlevel, corresponde al estado en el cual se encuentra Unix/Linux. Init controla este estado. Cada estado dispone de su propia configuración (o por inittab, o por scripts llamados initscripts). Por ejemplo, se puede utilizar un nivel de ejecución para arrancar Unix en modo monousuario, en multiusuarios, con o sin red, con o sin modo gráfico. El administrador puede personalizar todos los niveles

Runlevels (Niveles de ejecución) son usados para customizar la forma en la que el SO arranca.

Runlevels nos indica qué servicios son iniciados automáticamente.

Cada **runlevel** tiene un identificador asociado

101.3: Niveles de Ejecución (Peso: 3)

CLASICO-UNIX

0: Parada del sistema

1: Modo *single-user.* No inicia servicios de red

2: Multi-user sin NFS

3: Full multi-user

4: No suele ser relevante y no se usa

5: Multiuser con X11.

6: Reinicio del Sistema

101.3: Niveles de Ejecución (Peso: 3)

DEBIAN-UBUNTU

0: Parada del sistema

1: Modo *single-user.* No inicia servicios de red

2: Multi-user con X11

3: Multi-user con X11

4: Multi-user con X11

5: Multiuser con X11.

6: Reinicio del Sistema

101.3: Niveles de Ejecución (Peso: 3)

SysV

El runlevel por defecto se especifica en el fichero

/etc/inittab

El SO carga los servicios según el runlevel cogiendo la tabla de **/etc/rcX.d/** dónde **X** es el identificador del runlevel

K: identificador de parada del servicio

S: identificador para que inicie el servicio

101.3: Niveles de Ejecución (Peso: 3)

Para iniciar el SO en función de un runlevel podemos ejecutar los comandos **init** o **telinit**

Ejemplo:

*¿Qué hará el comando **init 5**? ¿E **init 1**?*

101.3: Niveles de Ejecución (Peso: 3)

Niveles en CentOS / Red Hat

En Red Hat/Fedora y openSUSE, el comando **chkconfig** permite añadir, suprimir, activar o desactivar scripts, por nivel de

ejecución.

chkconfig [opción] [servicio]

Aunque la sintaxis sea idéntica en las dos distribuciones, chkconfig no funciona de la misma manera. En Red Hat, se inserta una línea especial al principio del script que indica a chkconfig los parámetros por defecto **(runlevels, posiciones de inicio y parada).** En openSUSE, chkconfig es un frontend para el comando **insserv**. Este último aprovecha también el encabezamiento de los scripts, pero de manera más compleja (gestiona la ordenación y el paralelismo, por ejemplo).

101.3: Niveles de Ejecución (Peso: 3)

Niveles en CentOS / Red Hat

Presentamos las primeras líneas del script de inicio de servicio sshd en Red Hat. El script se inicia y se para en los niveles 2, 3,

4 y 5. Se ejecuta en posición 55 (S55sshd) y se para en posición 25 (K25sshd).

chkconfig: 2345 55 25

description: OpenSSH server daemon

101.3: Niveles de Ejecución (Peso: 3)

Opciones chkconfig (Red Hat /CentOS)

list: lista del conjunto de la configuración.

list servicio: la configuración de un servicio dado.

add servicio: añade el servicio indicado en la configuración

del servicio: suprime el servicio de la configuración System V.

level xxx servicio **on/off**: activa o desactiva el servicio para los niveles de ejecución indicados

101.3: Niveles de Ejecución (Peso: 3)

Init en Debian

El comando **update-rc.d** crea los vínculos necesarios en los distintos directorios rcn.d, un poco como **chkconfig**, pero de manera más «bruta»: los scripts no tienen la información particular en su encabezamiento y el comando sólo funciona a nivel del sistema de ficheros, por lo que instala los diferentes vínculos según sus indicaciones.

101.3: Niveles de Ejecución (Peso: 3)

Init en Debian

Veamos un ejemplo de como se añade un servicio con parámetros por defecto. En este caso, se configura el servicio para ejecutarse en los niveles de 2 a 5 y pararse a los niveles 0, 1 y 6.

 # update-rc.d ssh defaults

101.3: Niveles de Ejecución (Peso: 3)

SysV – Systemd

La versión de Red Hat 7.0 en adelante (o CentOS) ha incluído Systemd como gestión de arranque y servicios gestionados usando para la gestión de los demonios y niveles de ejecución el comando:

systemctl

101.3: Niveles de Ejecución (Peso: 3)

Sysvinit Command	Systemd Command	Notes
service frobozz start	systemctl start frobozz	Inicia un servicio
service frobozz stop	systemctl stop frobozz	Para un servicio
service frobozz restart	systemctl restart frobozz	Reinicia un servicio
service frobozz reload	systemctl reload frobozz	Siempre que esté soportado, recarga la configuración de un servicio.
service frobozz condrestart	systemctl condrestart frobozz	Reinicia un servicio arrancado previamente
service frobozz status	systemctl status frobozz	Devuelve el estado del servicio
ls /etc/rc.d/init.d/	systemctl (or) systemctl list-unit-files --type=service (or) ls /lib/systemd/system/*.service /etc/systemd/system/*.service	Listado de los servicios que pueden ser iniciados o parados.
chkconfig frobozz on	systemctl enable frobozz	Inicio de servicio con el equipo
chkconfig frobozz off	systemctl disable frobozz	No iniciar el servicio con el equipo
chkconfig frobozz	systemctl is-enabled frobozz	Chequea si el servicio está o no listo para ser iniciado con el servidor
chkconfig --list	systemctl list-unit-files --type=service (or) ls /etc/systemd/system/*.wants/	Lista toda la configuración de procesos automáticos
chkconfig frobozz --list	ls /etc/systemd/system/*.wants/frobozz .service	Lista los runlevels
chkconfig frobozz --add	systemctl daemon-reload	Añade el servicio a la configuración de servicios automáticos.

101.3: Niveles de Ejecución (Peso: 3)

Mientras que en SysV podemos ver el nivel de arranque por el comando "runlevel" en Systemd usaremos:

Systemctl list-units –type=target

o **runlevel**

Para modificarlo:

systemctl set-default multi-user.target

Si queremos forzar la modificación:

systemctl set-default -f multi-user.target

Tiene el mismo efecto que colocar un lynk simbolico hacia:

/etc/systemd/system/default.target

Comando sysvinit	Comando systemd	Notas
telinit 3	systemctl isolate multi-user.target (OR systemctl isolate runlevel3.target OR telinit 3)	Cambia nivel de ejecución a multiusuario.
sed s/^id:.*:initdefault:/id:3:initdefault:/	ln -sf /lib/systemd/system/multi-user.target /etc/systemd/system/default.target	Define el nivel de ejecución multiusuario para el próximo reinicio del sistema.

101.3: Niveles de Ejecución (Peso: 3)

SysV Runlevel	systemd Target	Notes
0	runlevel0.target, poweroff.target	Parada del sistema
1, s, single	runlevel1.target, rescue.target	Modo Single User
2, 4	runlevel2.target, runlevel4.target, multi-user.target	Definido por usuario
3	runlevel3.target, multi-user.target	Multi-usuario sin entorno gráfico
5	runlevel5.target, graphical.target	Multi-usuario con entorno gráfico
6	runlevel6.target, reboot.target	Reboot
emergency	emergency.target	Emergency shell

101.3: Niveles de Ejecución (Peso: 3)

Otros comandos

top → carga de Servicios

ps → servicios corriendo en el sistema

pstree

w → carga del sistema

kill → mandar una señal al proceso

killall

bg → background

fg → foreground

pidof → saber el PID

101.3: Niveles de Ejecución (Peso: 3)

Otros comandos

El comando **shutdown** puede ser usando para apagar el sistema, reinciarlo o simplemente mandar mensajes de aviso a los usuarios conectados.

Ejemplos

shutdown –h +5 El sistema se parará en 5 min.

Para reiniciar el sistema:

shutdown –r now

101.3: Niveles de Ejecución (Peso: 3)

Mandar mensaje de aviso:

shutdown –k Desloguearos por favor

Programar hora de parada:

shutdown -h 23:59 #Paramos a las 23:59

101.3: Niveles de Ejecución (Peso: 3)

<u>De cara al examen:</u>

Describir partes esenciales de la BIOS

Describir los archivos que contienen información importante de HW

Gestionar Niveles de Ejecución

101: Arquitectura del Sistema

Preguntas de exámen:

¿Qué comando utilizarías para ver los mensajes que el Kernel ha dado durante la salida?